STANDALONE INTEGRATION HANDBOOK

Open Crypto Checkout

A practical guide to installing, configuring, integrating, testing, and deploying the non-custodial TypeScript crypto payment starter.

Security boundary: This package contains no private keys and cannot transfer funds. Customers pay merchant-controlled public addresses. The merchant's server remains responsible for order creation and fulfilment.

Version 1.0.0 · Requires Node.js 22+, pnpm 10, and PostgreSQL

1. Prerequisites

The receiving project must provide its own infrastructure and credentials:

- Node.js 22 or newer and pnpm 10
- A dedicated PostgreSQL database
- Public receiving addresses for every enabled network
- Reliable RPC/API providers for each blockchain
- A fiat-to-crypto rate service
- Public HTTPS domains for the API and checkout
- A secret manager for database and merchant credentials

Never transfer a populated `.env` file. Every recipient must create their own database, wallets, provider accounts, API key, and webhook secret.

2. Extract and install

```
tar -xzf open-crypto-checkout-v1.0.0.tar.gz
cd open-crypto-checkout

corepack enable
corepack prepare pnpm@10.15.1 --activate
pnpm install --frozen-lockfile
```

Confirm that `node --version` reports Node 22 or newer.

Repository layout

```
apps/
  api/       Express API and persisted background workers
  web/       React/Vite checkout and development storefront
packages/
  core/      Payment contracts, statuses and rail registry
  verifiers/ EVM, Bitcoin, Solana and Tron verification
examples/
  merchant-express/  Signed webhook receiver example
```

3. Create the database

Create a PostgreSQL database dedicated to the checkout and a least-privilege application user. Configure its URL through the target platform's secret manager.

```
cp .env.example .env

# Local example only
DATABASE_URL=postgresql://crypto_user:password@localhost:5432/crypto_payments

pnpm --filter @open-crypto-checkout/api db:migrate
```

For production, use TLS, automated backups, point-in-time recovery, monitoring, restricted network access, and a tested restoration procedure.

4. Configure receiving wallets

```
ETHEREUM_WALLET=0xYourEthereumAddress
BSC_WALLET=0xYourBscAddress
ARBITRUM_WALLET=0xYourArbitrumAddress
POLYGON_WALLET=0xYourPolygonAddress
SOLANA_WALLET=YourSolanaAddress
TRON_WALLET=YourTronAddress
BITCOIN_WALLET=bc1YourBitcoinAddress
```

Never configure seed phrases or private keys. Only public receiving addresses belong in this service. Independently verify every address and make a small test transfer on every enabled network.

5. Configure blockchain providers

```
ETHEREUM_RPC_URL=https://your-ethereum-provider.example
BSC_RPC_URL=https://your-bsc-provider.example
ARBITRUM_RPC_URL=https://your-arbitrum-provider.example
POLYGON_RPC_URL=https://your-polygon-provider.example
SOLANA_RPC_URL=https://your-solana-provider.example
TRON_RPC_URL=https://your-tron-provider.example
BITCOIN_RPC_URL=https://your-bitcoin-esplora-provider.example
```

Network	Assets	Default finality
Ethereum	ETH, USDT, USDC	12 confirmations
BNB Smart Chain	USDT	15 confirmations
Arbitrum	USDT, USDC	20 confirmations
Polygon	USDT, USDC	128 confirmations
Solana	USDT, USDC	Finalized
Tron	USDT	19 confirmations

Bitcoin	BTC	3 confirmations

Use reliable commercial providers for production and confirm their responses match the raw adapters under `packages/verifiers`.

6. Implement the rate provider

Production requires `RATE_PROVIDER_URL`. The API sends:

```
POST /quote
Content-Type: application/json

{
  "fiatAmount": "49.99",
  "fiatCurrency": "USD",
  "railId": "ethereum-usdc"
}
```

The service must respond with a positive decimal rate, source, and timestamp:

```
{
  "rate": "1.0002",
  "source": "merchant-rate-service",
  "quotedAt": "2026-09-06T17:00:00.000Z"
}
```

The rate is the fiat price of one whole crypto asset. The API performs exact decimal-to-base-unit conversion. Do not calculate payment amounts with floating-point arithmetic.

7. Configure policies and origins

```
NODE_ENV=development
PORT=8090
WEB_ORIGIN=http://localhost:5173
PUBLIC_API_ORIGIN=http://localhost:8090
PUBLIC_CHECKOUT_ORIGIN=http://localhost:5173

QUOTE_TTL_SECONDS=900
UNDERPAY_TOLERANCE_BPS=100
OVERPAY_REVIEW_BPS=200
```

The defaults provide a 15-minute quote, approximately 1% underpayment tolerance, and manual review above 2% overpayment. Review these values for the merchant's risk model.

8. Configure development credentials

```
BOOTSTRAP_MERCHANT_KEY=replace-with-a-long-random-value
BOOTSTRAP_WEBHOOK_SECRET=replace-with-a-different-random-value
BOOTSTRAP_WEBHOOK_URL=http://localhost:8091/webhooks/crypto
ALLOW_LOCAL_WEBHOOKS=true
```

Generate separate random values, for example with `openssl rand -hex 32`. Development bootstrap is server-side and idempotent. The browser never receives the merchant key.

In production, use HTTPS and set `ALLOW_LOCAL_WEBHOOKS=false`.

9. Start development

```
pnpm --filter @open-crypto-checkout/api db:migrate
pnpm dev
```

Defaults:

- Checkout: `http://localhost:5173`
- API: `http://localhost:8090`

```
curl http://localhost:8090/health
curl http://localhost:8090/ready
```

The deterministic development rate exists only for interface testing and must never be used for real payments.

10. Create payments from the merchant server

Never create a payment directly from browser code. The merchant backend must use its server-only bearer key and a stable idempotency key.

```
const response = await fetch(
  `${process.env.CRYPTO_PAYMENT_API_URL}/v1/payments`,
  {
    method: "POST",
    headers: {
      authorization: `Bearer ${process.env.CRYPTO_PAYMENT_API_KEY}`,
      "content-type": "application/json",
      "idempotency-key": `crypto-payment:${order.id}`,
    },
    body: JSON.stringify({
      merchantOrderReference: order.id,
      fiatAmount: order.total,
      fiatCurrency: order.currency,
      rails: [
        "ethereum-eth", "ethereum-usdt", "ethereum-usdc",
        "bsc-usdt", "arbitrum-usdt", "arbitrum-usdc",
        "polygon-usdt", "polygon-usdc",
        "solana-usdt", "solana-usdc",
        "tron-usdt", "bitcoin-btc"
      ],
    }),
  }
);

if (!response.ok) throw new Error("Payment creation failed");
const payment = await response.json();
```

The order total must be loaded from the merchant database. Never trust an amount supplied by the browser. Redirect the customer to the returned checkout URL.

Idempotency

Retries for the same order must reuse the same key, such as `crypto-payment:order-1001`. Generating a new key on every retry can create duplicate payment requests.

11. Customer checkout flow

1. `GET /v1/checkout/:publicId` loads restricted checkout data.
2. `POST /v1/checkout/:publicId/select` with `{"railId":"ethereum-usdc"}` locks the quote.
3. The checkout displays the exact network, asset, destination, amount, and expiry.
4. The customer sends funds from their wallet.
5. `POST /v1/checkout/:publicId/transactions` submits `{"transactionHash":"0x..."}`.

The server—not the browser—owns the wallet, token, network, amount, tolerance, finality target, and quote expiry. The submitted transaction is bound to that immutable quote. The same on-chain transaction cannot pay multiple requests on the same chain, even across different tokens.

12. Payment states

```
created → awaiting_payment → transaction_submitted → confirming → paid
```

Terminal or review states also include expired, underpaid, overpaid review, failed, and cancelled. Fulfil only after the authoritative state is `paid`; neither submission nor confirmation-in-progress is sufficient.

13. Verify signed webhooks

The service sends the exact JSON bytes with:

- `x-webhook-timestamp` — Unix seconds
- `x-webhook-signature` — lowercase hex HMAC-SHA256 of `timestamp + "." + rawBody`

In Express, capture the untouched body before JSON parsing:

```
app.post(
  "/webhooks/open-crypto-checkout",
  express.raw({ type: "application/json", limit: "100kb" }),
  async (req, res) => {
    const event = verifier.verify({
      rawBody: req.body,
      timestamp: req.header("x-webhook-timestamp"),
      signature: req.header("x-webhook-signature"),
    });

    await database.transaction(async (tx) => {
      const inserted = await tx.events.insertIfMissing(event.id);
      if (!inserted) return;

      if (event.type === "payment.paid") {
        await tx.orders.markPaid(event.merchantOrderReference);
      }
    });

    res.status(204).end();
  }
);
```

Use a unique database constraint on the event ID. Deduplication and fulfilment must occur in the same transaction. The included example under `examples/merchant-express` demonstrates signature verification.

14. Confirm status independently

Webhooks are retryable and must not be the sole source of truth. The merchant backend can query:

```
GET /v1/payments/:publicId
Authorization: Bearer <merchant key>
```

Recommended fulfilment sequence:

1. Verify and deduplicate the signed webhook.
2. Query the authoritative payment status.
3. Confirm the merchant order reference and expected amount.
4. Mark the store order paid atomically.
5. Begin fulfilment.

> Never fulfil because the browser shows success, a wallet says “submitted,” a customer provides a screenshot, or a transaction hash merely exists.

15. Build and test

```
pnpm check:standalone
pnpm test:clean
pnpm typecheck
pnpm build
```

Run PostgreSQL concurrency tests against a disposable database:

```
export
TEST_DATABASE_URL="postgresql://test_user:test_password@localhost:5432/crypto_payments_test"
pnpm test
```

Never point integration tests at production. Confirm that the PostgreSQL tests run with zero skips before accepting real payments.

16. Production topology

```
shop.example.com
  Existing merchant storefront and order service

checkout.example.com
  Built React checkout

payments-api.example.com
  Express API, verification loop and webhook loop

Managed PostgreSQL
  Payments, quotes, transactions, jobs and webhooks
```

Production startup requires all RPC URLs and a rate provider. On shutdown, the service stops claiming new jobs, waits for active verification and webhook operations, closes HTTP, and then closes PostgreSQL.

17. Production security checklist

- ☐ Node.js 22 is used.
- ☐ All tests pass, including PostgreSQL tests with zero skips.
- ☐ Every wallet address and token contract/mint is independently verified.
- ☐ Small live transfers are tested on every enabled rail.
- ☐ RPC reliability, rate limits, monitoring, and finality assumptions are reviewed.
- ☐ Rate-provider failure stops quote creation rather than falling back silently.
- ☐ The merchant API key exists only on the server.
- ☐ The webhook secret is separate from the merchant key.
- ☐ Webhook event IDs are durably deduplicated.
- ☐ The store checks authoritative paid status before fulfilment.
- ☐ HTTPS, strict CORS, rate limits, and least-privilege access are enabled.
- ☐ `ALLOW_LOCAL_WEBHOOKS` is disabled.
- ☐ Database backup and restoration are tested.
- ☐ Alerts cover failed verification and webhook jobs.
- ☐ An incident-response procedure is documented.
- ☐ Independent security, legal, and tax reviews are complete.

> **Core integration rule:** the merchant server creates the payment; the checkout collects the customer's transaction hash; and the merchant fulfils only after a verified signed webhook and authoritative `paid` status.